Let’s see how it works. Open a Terminal window and navigate to the root folder of your Catalog Service project (catalog-service). Then use the `kubeval` command to validate the Kubernetes manifests within the k8s directory (`-d k8s`). The `--strict` flag disallows adding additional properties not defined in the object schema:

```
$ kubeval --strict -d k8s

PASS - k8s/deployment.yml contains a valid Deployment (catalog-service)
PASS - k8s/service.yml contains a valid Service (catalog-service)
```

In the next section, you’ll see how to use Kubeval in the commit stage workflow we implemented with GitHub Actions.

7.6.2 Automating Kubernetes manifests validation with GitHub Actions

GitHub Actions is the workflow engine we used to implement the commit stage for the Catalog Service’s deployment pipeline. Let’s extend it to include the Kubernetes manifest validation step, as illustrated in figure 7.15.

Go to your Catalog Service project (catalog-service), and open the commit-stage.yml file within the .github/workflows folder. To implement the validation step, we’ll rely on an action built by Stefan Prodan. He’s the maintainer of FluxCD, a CNCF-incubating project providing a continuous deployment solution on Kubernetes based on the GitOps principles. The action lets you install specific versions of useful Kubernetes-related tools. We’ll configure the action to install kubectl and Kubeval.

Listing 7.7 Validating the Kubernetes manifests for Catalog Service

```
name: Commit Stage
on: push
...
```